10

Real-time com WebSockets

Laravel na prática

Atualizações ao vivo com autorização, presença e estado previsível.

Roteiro da aula

- Problema real no projeto base
- WebSocket em linguagem simples
- Laravel Reverb
- Broadcasting
- Canais privados
- Presence Channels
- Laravel Echo
- Notificações multicanal
- Projeto do módulo

Problema real no projeto base

A recepção precisa ver novos agendamentos, chegadas e mudanças de status sem atualizar a página.

- O desafio não termina na entrega do evento. O sistema precisa autorizar cada canal, evitar dados de outras clínicas e atualizar a interface sem duplicar registros ou perder o estado atual.

WebSocket em linguagem simples

HTTP comum
Cada atualização depende de uma nova requisição do cliente.

WebSocket
Cliente e servidor mantêm uma conexão para trocar eventos em baixa latência.

Quando usar

- Use em mudanças que precisam aparecer imediatamente. Para dados ocasionais, uma requisição HTTP continua mais simples.

Laravel Reverb

Função

Reverb executa o servidor WebSocket compatível com o protocolo Pusher dentro do ecossistema Laravel.

- Operação
- O processo precisa rodar separadamente da aplicação HTTP e ficar atrás de um proxy compatível com conexões persistentes.

Exemplo de código

```
php artisan install:broadcasting
php artisan reverb:start

BROADCAST_CONNECTION=reverb
REVERB_APP_ID=clinicflow
REVERB_APP_KEY=local-key
REVERB_APP_SECRET=local-secret
REVERB_HOST=localhost
REVERB_PORT=8080
```

Broadcasting

Responsabilidade

O evento descreve o que aconteceu e escolhe o canal de destino.

Entrega

Use ShouldBroadcast quando a resposta HTTP puder aguardar. Use ShouldBroadcastNow somente quando a publicação imediata for realmente necessária.

Exemplo de código

```
final class AppointmentCreated implements ShouldBroadcast
{
    public function broadcastOn(): Channel
    {
        return new PrivateChannel(
            "clinic.{$this->appointment->clinic_id}"
        );
    }

    public function broadcastAs(): string
    {
        return 'appointment.created';
    }
}
```

Canais privados

Regra

O servidor autentica o usuário e autoriza o acesso ao canal solicitado.

- Proteção
- A autorização deve comparar a clínica do usuário com o identificador do canal. Nunca confie apenas no nome enviado pelo frontend.

Exemplo de código

```
Broadcast::channel('clinic.{clinicId}',
    function (User $user, int $clinicId): bool {
        return $user->clinic_id === $clinicId;
    }
);

Echo.private(`clinic.${clinicId}`)
    .listen('.appointment.created', refreshAgenda);
```

Presence Channels

Diferença

Presence channel aplica a mesma autorização do canal privado e também informa quem entrou, saiu ou já estava conectado.

- Uso responsável
- Compartilhe somente dados mínimos do usuário. Presença ajuda na colaboração, mas não deve substituir bloqueios e controle de concorrência.

Exemplo de código

```
Broadcast::channel('agenda.{clinicId}',
    function (User $user, int $clinicId): array|false {
        if ($user->clinic_id !== $clinicId) {
            return false;
        }

        return [
            'id' => $user->id,
            'name' => $user->name,
        ];
    }
);
```

Laravel Echo

Responsabilidade

Echo assina canais e transforma eventos recebidos em mudanças de interface.

Estado previsível

Atualize a consulta ou o item afetado. Evite inserir registros sem verificar se eles já existem na tela.

Exemplo de código

```
Echo.private(`clinic.${clinicId}`)
  .listen('.appointment.created', ({ appointment }) => {
    queryClient.setQueryData(['appointments'], current =>
      mergeAppointment(current, appointment)
    )
  })

return () => Echo.leave(`clinic.${clinicId}`)
```

Notificações multicanal

Escolha de canal

- O tipo e a urgência da mensagem definem os canais. Broadcast atende a interface ao vivo, database mantém histórico e mail alcança quem está fora do sistema.
- Consistência
- Centralize a decisão na Notification para evitar regras diferentes em cada controller ou job.

Exemplo de código

```
final class UrgentAppointment extends Notification
{
    public function via(object $notifiable): array
    {
        return ['database', 'broadcast', 'mail'];
    }

    public function toArray(object $notifiable): array
    {
        return [
            'appointment_id' => $this->appointment->id,
            'message' => 'Novo agendamento urgente',
        ];
    }
}
```

Projeto do módulo

Missão

Atualizar a agenda em tempo real quando um agendamento for criado, cancelado ou mudar de status.

Critérios de conclusão

- Reverb roda como processo separado. O backend publica em canal privado autorizado. O frontend atualiza sem duplicar itens e encerra a assinatura ao sair da tela. O projeto registra uma notificação no banco e também a transmite por broadcast.

Checklist final

Ao final, o aluno deve conseguir
- explicar quando WebSocket agrega valor;
- configurar e executar o Reverb;
- publicar eventos com Broadcasting;
- autorizar canais privados e de presença;
- atualizar o estado da interface com Echo;
- evitar assinaturas e registros duplicados;
- escolher canais adequados para notificações.